

ETMS Project Documentation

Version: 1.0.0 Last updated: 2026-05-04 Source of truth: repository files only (no external assumptions)

Table of Contents

1. Executive Summary
2. Project Architecture
3. Technology Stack
4. Repository and Folder Structure
5. Getting Started
6. Environment Configuration
7. Database Documentation
8. Backend API Documentation
9. User Roles and Permissions
10. Business Logic and Workflows
11. Frontend Documentation
12. Core Services
13. Middleware and Utilities
14. Data Models and Type Definitions
15. Security
16. File Management
17. Background Jobs
18. Testing
19. Deployment
20. Configuration Reference
21. Troubleshooting
22. API Integration Guide
23. Data Import and Export
24. Maintenance and Operations
25. Development Guidelines
26. Recent Updates and Fixes
27. Future Enhancements
28. Appendices

1. Executive Summary

ETMS (Employee Ticket Management System) is a full-stack system for raising, approving, assigning, tracking, and resolving IT and operational tickets across departments. It includes role-based workflows, asset tracking, approval routing, SLA monitoring, and a dedicated Data Portal for data requests. It is built with React + Vite on the frontend and Node.js + Express + TypeScript on the backend, backed by PostgreSQL.

Key capabilities include:

- Multi-step ticket creation with categories, priorities, SLA days, and file attachments.

- Manager approval workflows with re-approval after escalation.
- Round-robin ticket assignment within a category/team.
- Asset management with ownership history and spec details.
- Data Portal for data-specific tickets and file downloads.
- SLA escalation job with email notifications.

Current status (per repository documentation): the backend is production-ready with a centralized Manager Service, access control enhancements, and SLA escalation tests passing.

2. Project Architecture

2.1 High-Level Architecture

- Client (React + Vite) communicates with the Server (Express API) using JSON and multipart requests.
- Server uses PostgreSQL with raw SQL (no ORM) and maintains strict role-based access control.
- Files are stored on disk (uploads/ and uploads/responses/) and referenced in the database.
- Email notifications are sent via SMTP or Ethereum in local development.
- A cron job runs SLA escalation checks.

2.2 Major Components

- Frontend: single-page app with role-specific routes and navigation.
 - Backend API: REST endpoints for auth, tickets, approvals, assets, categories, users, reports, data portal, and imports.
 - Database: normalized schema with ticketing, assets, attachments, logs, and permissions.
 - Background Job: SLA escalation using node-cron.
-

3. Technology Stack

Frontend

- React 18
- Vite 5
- TypeScript 5
- Tailwind CSS 3
- Axios
- React Router DOM
- date-fns

Backend

- Node.js
- Express
- TypeScript
- PostgreSQL (pg)
- JWT (jsonwebtoken)
- bcrypt

- multer (file upload)
- nodemailer
- node-cron
- helmet, cors, morgan

Database

- PostgreSQL
 - SQL migrations and seed scripts (database/*.sql)
-

4. Repository and Folder Structure

Top-level files:

- README.md: project overview and primary instructions
- README_BACKEND_FIXES.md: backend improvements summary
- DEPLOYMENT_CHECKLIST.md: production deployment checklist
- MANAGER_SERVICE_IMPLEMENTATION_COMPLETE.md: manager service summary
- SLA_ESCALATION_TEST_RESULTS.md: SLA job test results

Main workspace:

- etms-mvp/
 - client/: React frontend
 - server/: Express backend
 - database/: schema, seed, and migrations
 - scripts/: helper scripts
-

5. Getting Started

5.1 Prerequisites

- Node.js 18+
- PostgreSQL 14+
- npm

5.2 Database Setup

1. Create database: `psql -U postgres -c "CREATE DATABASE uiicdb;"`
2. Apply schema: `psql -U postgres -d uiicdb -f etms-mvp/database/schema.sql`
3. Seed data: `psql -U postgres -d uiicdb -f etms-mvp/database/seed.sql`

5.3 Backend

- Install: `cd etms-mvp/server && npm install`
- Run dev server: `npm run dev`
- Build: `npm run build`
- Typecheck: `npm run typecheck`

5.4 Frontend

- Install: `cd etms-mvp/client && npm install`
- Run dev server: `npm run dev`
- Build: `npm run build`
- Typecheck: `npm run typecheck`

5.5 Run Client and Server Together

From `etms-mvp/`: `npm run dev`

6. Environment Configuration

6.1 Server Environment Variables

Required (validated at startup):

- `DB_HOST`
- `DB_PORT`
- `DB_NAME`
- `DB_USER`
- `DB_PASSWORD`
- `JWT_SECRET` (recommended length ≥ 32)

Optional:

- `PORT` (default 5000)
- `CLIENT_URL` (default `http://localhost:5173`)
- `UPLOAD_DIR` (default `./uploads`)
- `RESPONSE_UPLOAD_DIR` (default `./uploads/responses`)
- `SMTP_HOST`
- `SMTP_PORT` (default 587)
- `SMTP_USER`
- `SMTP_PASS`
- `SMTP_SECURE` (true/false)
- `MAIL_FROM` (default "ETMS noreply@uiic.co.in")
- `SLA_CRON_SCHEDULE` (default `0 0 * * *`)

6.2 Client Environment Variables

- `VITE_API_URL` (default `http://localhost:5000/api`)
-

7. Database Documentation

7.1 ENUM Types

- `user_role_enum`: employee, manager, admin, data_team
- `priority_enum`: low, medium, high, critical

7.2 Core Tables

1. users

- id, emp_id, name, email, password_hash, role, category_id, team, is_active
- password_changed_at used to force password change on first login

2. ticket_types

- complaint, request, data

3. ticket_categories

- ticket_type_id, name, category_key
- default_priority, manager_user_id
- assigned_team_key, is_team, requires_approval

4. assets

- name, serial_number, category_id, assigned_to, status
- spec columns: machine_type, model, ram, hdd, monitor_serial, monitor_make, system_ip, port, ms_office_ver, os, host_id, floor, branch

5. asset_assignments

- ownership history for assets
- constraint ensures returned_at >= assigned_at
- unique index ensures only one active assignment per asset

6. tickets

- ticket_no (UIC-{C|R|D}-YYYY-XXXXXX)
- title, description, ticket_type_id, category_id, priority, status
- raised_by, assigned_to, approval_owner_id, asset_id
- rejection_reason, report_reason
- sla_days, sla_due_date, escalated, escalated_at

7. attachments

- ticket_id, filename, original_name, file_path, file_size, mime_type, uploaded_by, uploaded_at

8. ticket_logs

- audit trail with action, old_status, new_status, performed_by, note

7.3 Manager Assignment History

manager_assignment_history

- category_id, manager_user_id, action (assigned/removed), assigned_by, assigned_at, reason

7.4 Permissions System

- permissions
- role_permissions
- user_permissions
- functions: has_permission, get_user_permissions

7.5 Indexes and Constraints (highlights)

- tickets: indexes on raised_by, assigned_to, approval_owner_id, status, type, category, asset, sla
- assets: indexes on serial_number, category_id, assigned_to, status
- asset_assignments: active assignment unique index and history indexes

7.6 Seed and Migration Scripts

- schema.sql: base schema
- seed.sql: default team rows, users, categories, assets
- create_permissions_system.sql: permissions tables and functions
- create_manager_assignment_history.sql: history tracking
- patch_data_team_routing.sql: data team routing and schema backfills
- migrate_asset_spec_columns.sql: asset spec columns
- fix_categories.sql: replace categories with a specific 10-category model
- add_category_employees.sql: inserts employees for categories
- seed_two_managers.sql: two-manager variant

8. Backend API Documentation

Base URL: /api Auth: JWT via Authorization: Bearer

8.1 Authentication

- POST /api/auth/login
 - Body: { email, password }
 - Response: { token, user }
- GET /api/auth/me
 - Response: current user with role and permissions

8.2 Tickets

- POST /api/tickets
 - Access: employee
 - Body (multipart/form-data): title, description, ticket_type_id, category_id, priority, sla_days, asset_id (for hardware), file (optional)
- GET /api/tickets
 - Access: admin, manager, employee
 - Query: status, type, priority, ticket_id, page, limit
- GET /api/tickets/:id
- GET /api/tickets/:id/allowed-statuses
- PUT /api/tickets/:id/status
 - Body: { status, note?, report_reason? }

- GET /api/tickets/:id/file
- POST /api/tickets/:id/response-file
 - Access: employee/manager (assignee only) for data tickets
- GET /api/tickets/:id/response-file

8.3 Approvals

- GET /api/approvals/pending
- POST /api/approvals/:ticketId/approve
- POST /api/approvals/:ticketId/reject
- POST /api/approvals/:ticketId/reapprove

8.4 Assets

- GET /api/assets
- GET /api/assets/my
- GET /api/assets/:id
- GET /api/assets/:id/history
- POST /api/assets
- PATCH /api/assets/:id/spec
- PATCH /api/assets/:id/status
- POST /api/assets/:id/transfer
- DELETE /api/assets/:id

8.5 Users

- GET /api/users
- POST /api/users
- PATCH /api/users/change-password
- PATCH /api/users/:id/toggle-active
- PATCH /api/users/:id/name
- POST /api/users/transfer-ownership
- DELETE /api/users/:id

8.6 Categories and Teams

- GET /api/categories
- GET /api/categories/:categoryId/employees
- GET /api/categories/teams
- POST /api/categories/teams
- DELETE /api/categories/teams/:teamId

8.7 Reports

- GET /api/reports/top-failing-devices
 - Query: limit (1-50)

8.8 Lookup

- GET /api/lookup/employee?q=EMP_ID_OR_SERIAL

8.9 Import

- POST /api/import/assets/excel
 - Access: admin, manager
 - Body (multipart/form-data): file (.xlsx/.xls)

8.10 Data Portal

- GET /api/data-portal/teams
 - POST /api/data-portal/tickets
 - GET /api/data-portal/tickets
 - GET /api/data-portal/tickets/:id
 - GET /api/data-portal/tickets/:id/response-file
-

9. User Roles and Permissions

Roles:

- employee
- manager
- admin
- data_team

Access control is enforced by middleware and a permission system in the database. Key rules:

- Admin: full access, view-only on ticket status changes.
- Manager: access only to categories they manage; approvals and re-approvals.
- Employee: access to own tickets and assigned tickets; can raise tickets.
- Data Team: restricted to data portal workflow.

Assets:

- Only managers of infra_team can manage assets.
 - Employees can view their assets; infra employees can mark under_repair.
-

10. Business Logic and Workflows

10.1 Ticket Lifecycle

pending_approval -> approved -> assigned -> in_progress -> resolved -> closed

Escalation path:

- report (reported) returns to pending_approval for request tickets
- rejected and closed are terminal states

10.2 Approval Workflow

- Manager approves pending tickets.
- Approved tickets auto-assign to least-loaded employee in mapped team.
- Re-approval requires explicit assignee and applies to escalated tickets.

10.3 Assignment Logic

- Round-robin assignment based on open ticket count per employee.

10.4 Asset Management

- Assets can be created, updated, transferred, and tracked in history.
- Transfers create history rows and close previous assignments.

10.5 SLA Escalation

- Tickets in assigned or in_progress past SLA due date are escalated.
- Escalation emails sent to category manager.
- 24-hour re-escalation rule prevents repeated alerts.

10.6 Manager Assignment

- ManagerService centralizes manager operations with caching.
 - Assignment history is recorded to manager_assignment_history.
-

11. Frontend Documentation

11.1 Routes

- /login (public)
- /change-password (forced on first login)
- /dashboard
- /tickets
- /tickets/new (employee only)
- /tickets/:id
- /approvals (manager only)
- /reports (manager only)
- /assets (employee + infra manager)
- /admin (admin only)
- /admin/teams (admin only)
- /data-portal (data_team only)

11.2 Pages

- LoginPage: authentication flow with password change check.
- ChangePasswordPage: enforced for initial login.
- DashboardPage: KPIs and recent tickets by role.
- TicketsPage: role-scoped ticket list with filters.
- NewTicketPage: step-based ticket creation with asset selection for hardware complaints.
- TicketDetailPage: ticket details, logs, status actions, uploads.

- PendingApprovalsPage: manager re-approval queue.
- ReportsPage: top failing devices report with auto-refresh.
- AssetsPage: employee asset cards; manager asset table with actions; Excel import.
- AdminPage: user management, asset management, ticket list, lookup.
- AdminTeamsPage: team management, create/delete teams, members.
- DataPortalPage: data ticket creation and downloads.

11.3 Core Components

- AppLayout and Sidebar: consistent layout and role-based navigation.
- TicketCard, TypeBadge, StatusBadge, PriorityBadge: ticket visuals.
- TicketTypeSelector, CategorySelector: ticket creation steps.
- AssetDetailModal: spec viewing/editing.
- CategoryOrgGraph: manager/employee org visualization.

12. Core Services

- ManagerService: manager lookup, assignment history, workload metrics, caching.
- managerAssignmentService: legacy wrapper around ManagerService.
- accessControlService: centralized access control checks and permission logic.
- assignmentService: round-robin employee selection.
- emailService: notification templates and send wrappers.

13. Middleware and Utilities

Middleware

- authMiddleware: JWT verification and user load.
- roleMiddleware: role guard helper.
- uploadMiddleware: multer with allowed MIME types and size limits.

Utilities

- jwtUtils: JWT sign/verify (8h expiration).
- generateTicketId: ticket number generation.
- ticketTransitions: allowed status transitions and validation.
- sanitizeFilename: safe file naming.
- fileCleanup: safe deletion for error paths.
- validateEnv: environment validation on startup.

14. Data Models and Type Definitions

Backend types are in `server/types/index.ts` and mirror database rows. Frontend types are in `client/src/types/index.ts` and mirror API payloads.

Key entities:

- User
 - Ticket
 - Category
 - Asset
 - TicketLog
 - Attachment
-

15. Security

- JWT-based auth with 8h expiry.
 - bcrypt password hashing (10 rounds).
 - Environment variables validated at startup.
 - Helmet and CORS enabled.
 - Role-based access enforcement.
 - File uploads restricted by MIME type and size.
-

16. File Management

- Request attachments stored in uploads/.
 - Data response files stored in uploads/responses/.
 - Files are saved with sanitized names and tracked in attachments table.
 - Downloads are served via /uploads or direct response file endpoints.
-

17. Background Jobs

SLA Escalation Job:

- schedule: SLA_CRON_SCHEDULE or 0 0 * * *
 - logic: escalates tickets past SLA due date and sends manager emails
-

18. Testing

Repository scripts:

- server/scripts/testManagerService.ts
- server/scripts/testAccessControl.ts
- server/scripts/testSlaEscalation.ts
- server/scripts/testSlaEscalationViaServer.ts
- server/scripts/verifyManagerService.ts

Root scripts (etms-mvp/package.json):

- test:manager
- test:data-portal-routing
- test:ticket-cycle-consistency
- test:reported-transition

19. Deployment

See DEPLOYMENT_CHECKLIST.md for full steps. Highlights:

- Build backend and frontend.
 - Configure .env securely.
 - Ensure uploads and response directories exist.
 - Run database migrations.
 - Verify /health endpoint.
-

20. Configuration Reference

Backend

- PORT
- CLIENT_URL
- DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD
- JWT_SECRET
- SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, SMTP_SECURE
- MAIL_FROM
- UPLOAD_DIR
- RESPONSE_UPLOAD_DIR
- SLA_CRON_SCHEDULE

Frontend

- VITE_API_URL
-

21. Troubleshooting

Common issues:

- 401 Unauthorized: check token in sessionStorage and JWT_SECRET.
 - File upload errors: verify allowed MIME types and size limits.
 - Missing env vars: validateEnv will exit server on missing required vars.
 - Data tickets resolution: response file required before resolving.
-

22. API Integration Guide

1. Authenticate via POST /api/auth/login.
 2. Store token and include in Authorization header.
 3. For multipart endpoints, use multipart/form-data with file field name "file".
 4. Handle 401 globally (client auto-logout).
-

23. Data Import and Export

- Asset import via `/api/import/assets/excel`.
 - Data portal downloads via `/api/data-portal/tickets/:id/response-file`.
 - Ticket attachments via `/api/tickets/:id/file` and `/api/tickets/:id/response-file`.
-

24. Maintenance and Operations

- Monitor server logs for SLA job, email sending, and errors.
 - Regular DB backups and uploads directory backups are recommended.
 - Review `asset_assignments` and `ticket_logs` for audit trails.
-

25. Development Guidelines

- TypeScript strict mode used on both client and server.
 - Centralize role strings and transition logic in constants.
 - Use `accessControlService` for permission checks.
-

26. Recent Updates and Fixes

From `README_BACKEND_FIXES.md` and `MANAGER_SERVICE_IMPLEMENTATION_COMPLETE.md`:

- Environment variable validation added.
 - JWT expiration updated to 8 hours.
 - File cleanup and upload directory handling improved.
 - Manager Service centralized with caching and history.
 - Access control service introduced with RBAC support.
 - Health check endpoint added.
 - SLA escalation system tested and verified.
-

27. Future Enhancements

Not implemented, recommended in repo docs:

- Rate limiting
 - Swagger/OpenAPI
 - Structured logging
 - Workload balancing algorithms
 - More comprehensive tests
-

28. Appendices

28.1 Ticket Status Reference

- `pending_approval`
- `approved`
- `assigned`

- in_progress
- reported
- resolved
- closed
- rejected

28.2 Default Login Accounts (from seed.sql)

- admin@uiic.co.in
- mgr.infra@uiic.co.in
- mgr.network@uiic.co.in
- data.portal@uiic.co.in

28.3 File Upload Constraints

- Allowed MIME types: pdf, doc, docx, txt, png, jpg
- Max size: 5 MB (tickets), 10 MB (excel import)

End of document.